

项目 RAG（Retrieval-Augmented Generation）实现思路整理

一、RAG 架构总览

项目采用经典的 两阶段 RAG 架构：索引构建阶段 和 检索推理阶段，整体数据流如下：



二、核心组件详解

2.1 文档加载器 — document_loader.py

职责：将知识文档切分为可检索的文本块，保留语义完整性和上下文信息。

关键设计：

```
class DocumentLoader:
    @staticmethod
    def load_documents(base_dir: str = None) -> list[dict]:
        # 加载 knowledge_base/ 目录下的 .md 和 .txt 文件
        # 提取一级标题作为文档标题
        # 返回: [{"content": "...", "metadata": {"source": "文件名", "title": "标题"}}]
```

分块策略（split_documents 方法）：

参数	值	设计理由
chunk_size	500	每块约 500 字符，保证语义完整且不超过 Embedding 模型限制
chunk_overlap	50	相邻块重叠 50 字符，避免段落边界语义被截断

智能分块逻辑：

```
# 1. 按段落分割（保留语义完整性）
paragraphs = content.split("\n\n")

# 2. 追踪当前标题层级（保留上下文）
if para.startswith("#"):
    current_headers = [...] # 维护标题栈

# 3. 判断是否需要切分
if len(current_chunk) + len(para) > chunk_size:
    # 保存当前块，保留重叠部分
    current_chunk = current_chunk[-chunk_overlap:] + "\n\n" + para

# 4. 每个块携带标题上下文
header_context = "> ".join(h["text"] for h in current_headers)
```

元数据结构：

```
{
    "source": "yolo_basics.md",          # 文件名
    "title": "YOLO 基础概念",           # 文档标题（从 # 提取）
    "file_path": "/path/to/yolo_basics.md",
    "header_context": "YOLO 基础概念 > IoU", # 当前块所在的标题路径
    "chunk_index": 0,                    # 块序号
}
```

2.2 文本向量化服务 — embedding.py

职责：将文本转换为高维向量，支持通义千问和 OpenAI 两种 Embedding 模型。

模型选择策略：

优先级	模型	维度	API Base
1（优先）	通义千问 text-embedding-v3	1024	https://dashscope.aliyuncs.com/compatible-model/v1
2（兜底）	OpenAI text-embedding-3-small	1536	https://api.openai.com/v1

批量处理优化：

```
def embed_texts(self, texts: list[str]) -> list[list[float]]:
    # 分批处理，每批最多 20 条（避免 API 限流）
    batch_size = 20
    for i in range(0, len(texts), batch_size):
        batch = texts[i:i+batch_size]
        response = self._client.embeddings.create(
            model=self._model,
            input=batch,
        )
        batch_embeddings = [item.embedding for item in response.data]
        all_embeddings.extend(batch_embeddings)
```

关键函数：

方法	
embed_texts(texts)	批量向量化，返回向量列表
embed_query(query)	单个查询向量化，返回单个向量

2.3 Pgvector 向量存储客户端 — pgvector_client.py

职责：基于 PostgreSQL + pgvector 扩展实现向量存储和语义检索。

表结构设计：

```
CREATE TABLE knowledge_embeddings (
    id SERIAL PRIMARY KEY,
    content TEXT NOT NULL,           -- 文本内容
    metadata JSONB DEFAULT '{}',    -- 元数据（JSONB 格式）
    embedding vector(1024),         -- 向量（1024 维）
    created_at TIMESTAMP DEFAULT NOW()
);
```

索引策略：

```
CREATE INDEX idx_knowledge_embeddings_vector
ON knowledge_embeddings
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

参数	值	说明
<code>lists</code>	100	IVFFlat 索引的聚类数，适用于中等规模数据集
<code>ivfflat.probes</code>	10	检索时探测的聚类数，平衡精度与速度

检索实现：

```
def search(self, query_embedding, top_k=3):
    # 设置 probes 提高检索精度
    cur.execute("SET ivfflat.probes = 10;")

    # 使用 <=> 运算符计算余弦距离（返回相似度 = 1 - 距离）
    cur.execute("""
        SELECT content, metadata, 1 - (embedding <=> %s) as similarity
        FROM knowledge_embeddings
        ORDER BY embedding <=> %s
        LIMIT %s
        """,
        (query_embedding, query_embedding, top_k),
    )
```

关键函数：

方法	功能
<code>init_table()</code>	初始化表结构和索引
<code>insert_embeddings()</code>	批量插入向量数据
<code>search()</code>	语义检索，返回 top_k 条结果
<code>count()</code>	获取向量总数
<code>clear()</code>	清空向量表

2.4 知识检索器 — retriever.py

职责：将 RAG 流程串联为完整管道，提供统一的检索接口。

索引构建流程：

```
def build_index(self, force_rebuild: bool = False):
    # 1. 初始化 Pgvector 表
    pgvector_client.init_table()
```

```

# 2. 如果需要强制重建, 先清空
if force_rebuild:
    pgvector_client.clear()

# 3. 加载文档
documents = document_loader.load_documents()

# 4. 文本分块
chunks = document_loader.split_documents(documents)

# 5. 批量向量化
texts = [chunk["content"] for chunk in chunks]
embeddings = embedding_service.embed_texts(texts)

# 6. 过滤向量化失败的条目
valid_indices = [i for i, emb in enumerate(embeddings) if emb]

# 7. 存入 Pgvector
pgvector_client.insert_embeddings(valid_texts, valid_embeddings, valid_metadatas)

```

检索流程:

```

def search(self, query: str, top_k: int = 3) -> list[dict]:
    # 1. 确保索引已构建 (懒加载)
    if not self._index_built:
        self.build_index()

    # 2. 查询向量化
    query_embedding = embedding_service.embed_query(query)

    # 3. Pgvector 检索
    results = pgvector_client.search(query_embedding, top_k=top_k)

    return results

```

上下文格式化:

```

def format_context(self, results: list[dict]) -> str:
    # 格式化为 LLM 可用的上下文
    # [知识片段 1] (来源: xxx.md, 相似度: 0.95)
    # 内容...
    # ---
    # [知识片段 2] ...

```

2.5 知识库工具 — knowledge_tool.py

职责: 封装为 Agent 可调用的工具, 通过 @tool 装饰器注册。

相似度过滤机制:

```

@tool
def search_knowledge(query: str, top_k: int = 3) -> str:
    results = knowledge_retriever.search(query, top_k=top_k)

    if not results:
        return json.dumps({"answer": "知识库中暂无相关内容", "sources": []})

    # 相似度阈值过滤 (0.5)
    max_similarity = max(r.get("similarity", 0) for r in results)
    if max_similarity < 0.5:
        return json.dumps({"answer": "知识库中暂无相关内容", "sources": []})

    # 过滤低相似度结果
    formatted = []
    for r in results:
        if r.get("similarity", 0) >= 0.5:
            formatted.append({
                "content": r["content"][:300], # 截断到 300 字符
                "source": r.get("metadata", {}).get("source", "未知"),
                "similarity": r.get("similarity", 0),
            })

```

工具调用方式:

```

# 在 Agent 节点中调用
results_str = search_knowledge.invoke({"query": "什么是 IoU? ", "top_k": 3})
results = json.loads(results_str)

```

三、多 Agent 架构中的 RAG 集成

3.1 QA 节点 — nodes.py

```

def qa_node(state: Dict[str, Any]) -> Dict[str, Any]:
    """问答 Agent 节点: 处理知识问答任务"""
    messages = state["messages"]
    last_message = messages[-1].content

    # 调用知识检索工具
    results_str = search_knowledge.invoke({"query": last_message})
    results = json.loads(results_str)

    return {"qa_result": results}

```

3.2 Supervisor 汇总 — supervisor.py

知识结果处理:

```

def summarize(self, state: dict) -> dict:
    if state.get("qa_result"):

```

```
qa_result = state["qa_result"]
if isinstance(qa_result, dict) and "knowledge" in qa_result:
    # 有知识：构建知识上下文
    knowledge_parts = []
    for item in qa_result["knowledge"]:
        knowledge_parts.append(
            f"知识来源: {item.get('source', '未知')}, 相似度: {item.get('similarity', 0):.2f}\n"
            f"{item.get('content', '')}"
        )
    context_parts.append("\n---\n".join(knowledge_parts))
    has_knowledge = True
elif isinstance(qa_result, dict) and qa_result.get("answer") == "知识库中暂无相关内容":
    # 无知识：不添加知识上下文
    has_knowledge = False
```

Prompt 模板选择：

场景	Prompt 内容
有知识库	知识内容 + 对话历史 + 当前问题
有上下文（检测/分析结果）	上下文信息 + 对话历史 + 当前问题
无知识无上下文	对话历史 + 当前问题

四、知识库文档结构

当前知识库包含 3 个文档：

文档	内容
yolo_basics.md	YOLO 基础概念、IoU、NMS、损失函数等
evaluation_metrics.md	目标检测评估指标：mAP、Precision、Recall 等
remote_sensing.md	遥感图像特点、检测难点、数据集等

五、关键设计决策总结

设计点	选择	理由
文档分块	段落级优先，500 字符/块，50 字符重叠	保留语义完整性，避免边界截断
Embedding 模型	通义千问 text-embedding-v3 (1024 维)	中文支持更好，API 调用更稳定
向量数据库	PostgreSQL + pgvector	与项目主数据库统一，部署简单
索引类型	IVFFlat	中等规模数据集 (<10万条) 的最优选择
相似度阈值	0.5	过滤低质量检索结果，减少幻觉
检索结果数	top_k=3	平衡上下文长度与信息覆盖
知识注入位置	Supervisor 汇总阶段	统一管理所有上下文，保证回答一致性
检测请求处理	检测任务不查询知识库	避免无关检索，提高检测效率

六、完整调用链路




```
    "type": "text_chunk",
    "content": "YOLOv11 在以下方面进行了改进...",
    "knowledge_sources": [...],
    "has_knowledge": true
}
```

七、优化与改进建议

7.1 当前局限

- 1. 静态知识库：文档修改后需要重启服务重建索引
- 2. 单一向量检索：仅使用语义相似度，未结合关键词匹配
- 3. 固定阈值：相似度阈值 0.5 为硬编码，未根据查询动态调整
- 4. 无 Rerank：检索结果未经过重排序

7.2 潜在优化方向

优化项	实现思路
增量更新	监听文件系统变化，自动更新修改的文档块
混合检索	结合 BM25 关键词检索 + 向量检索，提升召回率
动态阈值	根据查询类型和检索结果分布动态调整阈值
Rerank	使用 Cross-Encoder 对检索结果重排序
多模态检索	支持图像 + 文本混合检索